

7.<Źródło>Optymalizacja stanowiska krytycznego

Zgodnie z Pańskim poleceniem, przedstawiam opracowanie zagadnienia "Optymalizacja obsługi zadań w stanowisku krytycznym" w oparciu o treść Rozdziału 9 załączonego dokumentu.

Optymalizacja Obsługi Zadań w Stanowisku Krytycznym

W teorii szeregowania, stanowisko krytyczne, czyli takie, które ogranicza przepustowość całego systemu produkcyjnego, jest często modelowane jako problem **jednomaszynowy**. Analiza tego przypadku stanowi fundament dla rozwiązywania bardziej złożonych zagadnień, a także jest bezpośrednio stosowana w optymalizacji pojedynczych, kluczowych etapów w systemach produkcyjnych.

Celem optymalizacji jest wyznaczenie optymalnego harmonogramu pracy stanowiska, czyli takiej kolejności wykonywania zadań, która minimalizuje przyjęte kryterium jakości. Harmonogram jest reprezentowany przez wektor terminów rozpoczęcia zadań $S = (S_1, \dots, S_n)$ lub, równoważnie, przez wektor terminów ich zakończenia $C = (C_1, \dots, C_n)$.

1. Problem Podstawowy i Proste Uogólnienia

W najprostszej formie, dla zbioru zadań $J = \{1, 2, \dots, n\}$ o znanych czasach operacji p_j , rozwiązanie sprowadza się do znalezienia optymalnej permutacji π tych zadań.

1.1. Minimalizacja Czasu Zakończenia Wszystkich Zadań (C_{max})

- **Problem z terminami gotowości ($1 \mid r_j \mid C_{max}$):** Każde zadanie j staje się dostępne do realizacji w chwili r_j (termin gotowości). Należy znaleźć harmonogram, który minimalizuje czas zakończenia ostatniego zadania (C_{max}). **Rozwiązanie optymalne** uzyskuje się poprzez zastosowanie **reguły Jacksona**: zadania szereguje się w kolejności niemalejących

terminów gotowości r_j . Odpowiada to zasadzie FIFO (First-In-First-Out). Złożoność obliczeniowa tego algorytmu wynosi $O(n \log n)$.

- **Problem z terminami dostarczenia ($1 \mid q_j \mid C_{max}$):** Po zakończeniu operacji zadania j na maszynie, wymaga ono dodatkowego czasu q_j na dostarczenie. Kryterium jest minimalizacja chwili, w której wszystkie zadania zostaną dostarczone. **Rozwiązanie optymalne** uzyskuje się szeregując zadania w kolejności nierosnących czasów dostarczenia q_j . Jest to przypadek szczególny reguły MWR (Most Work Remaining).

1.2. Minimalizacja Maksymalnego Spóźnienia (L_{max})

- **Problem z żądanymi terminami zakończenia ($1 \parallel L_{max}$):** Dla każdego zadania j określony jest żądany termin zakończenia d_j . Celem jest minimalizacja maksymalnego spóźnienia, zdefiniowanego jako $L_{max} = \max_j (C_j - d_j)$. **Rozwiązanie optymalne** dostarcza **reguła EDD (Earliest Due Date)**: zadania należy szeregować w kolejności niemalejących, żądanych terminów zakończenia d_j . Złożoność wynosi $O(n \log n)$.

2. Problem Ogólny i jego Złożoność

Problem jednomaszynowy staje się znacznie trudniejszy, gdy uwzględnimy jednocześnie terminy gotowości r_j oraz czasy dostarczenia q_j .

- **Problem ($1 \mid r_j, q_j \mid C_{max}$):** Jest to problem **silnie NP-trudny**. Oznacza to, że nie istnieje dla niego algorytm wielomianowy, a znalezienie rozwiązania optymalnego wymaga metod o złożoności wykładniczej. Stanowi on podstawę do oceny jakości dolnych ograniczeń w bardziej złożonych problemach, np. gniazdowych.

Wartość funkcji celu dla danej permutacji π można wyrazić nierekurencyjnie jako:

$$C_{max}(\pi) = \max_{1 \leq u \leq v \leq n} \left(r_{\pi(u)} + \sum_{s=u}^v p_{\pi(s)} + q_{\pi(v)} \right)$$

Wyrażenie to ma wygodną interpretację grafową, gdzie C_{max} odpowiada długości **drogi krytycznej** w odpowiednio skonstruowanym grafie

acyklicznym $G(\pi)$. Droga ta składa się z odcinków odpowiadających kolejnym operacjom na maszynie. Ciąg zadań na drodze krytycznej nazywany jest **blokiem**. Własności drogi krytycznej i bloków są kluczowe dla konstrukcji zarówno algorytmów dokładnych, jak i przybliżonych.

3. Algorytmy Przybliżone dla Problemu Ogólnego

Ze względu na NP-trudność problemu ogólnego, w praktyce stosuje się algorytmy przybliżone, które w czasie wielomianowym dostarczają rozwiązań o gwarantowanej jakości.

3.1. Algorytm 2-aproksymacyjny (Algorytm S / Schrage) Jest to najprostsza i bardzo intuicyjna technika. Jej zasada działania jest następująca: w każdej chwili, gdy maszyna staje się wolna, do wykonania kierowane jest to z dostępnych zadań (czyli takich, dla których r_j jest mniejszy lub równy aktualnemu czasowi), które ma największy czas dostarczenia q_j .

Przykład: Dla danych z Tabeli 9.2, algorytm S generuje permutację $\pi^S = (6, 1, 2, 3, 4, 5, 7)$ z wartością $C_{max}(\pi^S) = 53$. Wykres Gantta dla tego rozwiązania przedstawiono na Rysunku 9.3 w dokumencie.

3.2. Algorytmy o Lepszej Jakości Istnieją bardziej zaawansowane algorytmy, które oferują lepszy najgorszy przypadek niż algorytm S.

- **Algorytm 3/2-aproksymacyjny (NS):** Jest to szybki algorytm o złożoności $O(n \log n)$, który łączy regułę Jacksona z regułą Johnsona znaną z problemu dwumaszynowego. Porównuje on rozwiązanie z algorytmu S z rozwiązaniem uzyskanym przez specyficzne uporządkowanie zadań i zwraca lepsze z nich.
- **Algorytm 4/3-aproksymacyjny (HS):** Obecnie najlepszy znany algorytm aproksymacyjny dla tego problemu. Rozważa on problem podstawowy oraz jego "odwrotny" odpowiednik (z zamienionymi rolami r_j i q_j), generując dla nich serie rozwiązań i wybierając najlepsze. Jego złożoność jest jednak wyższa i wynosi $O(n^2 \log n)$.

4. Algorytmy Dokładne (Przeglądu)

Dla problemu NP-trudnego istnieją również algorytmy dokładne, które gwarantują znalezienie rozwiązania optymalnego. Opierają się one na schemacie **Podziału i Ograniczeń (B&B)**.

- **Algorytm Carliera:** Jest to elegancki algorytm typu B&B, który wykorzystuje własności drogi krytycznej i zadania interferencyjnego do tworzenia podproblemów i efektywnej eliminacji gałęzi w drzewie poszukiwań.
- **Algorytm Blokowy:** Alternatywne podejście, które bezpośrednio wykorzystuje grafową interpretację problemu. Definiuje ono zbiory kandydatów do przesunięcia na początek lub koniec bloku krytycznego, co pozwala na inteligentne generowanie i przeszukiwanie drzewa rozwiązań.

Algorytmy te, mimo wykładniczej złożoności, są w praktyce bardzo efektywne i potrafią rozwiązywać instancje problemu o rozmiarze nawet do 1000 zadań, co czyni je użytecznymi narzędziami pomocniczymi w analizie bardziej złożonych systemów.